ECE C147/C247, Winter 2026 **Homework #2**
Neural Networks & Deep Learning Prof. J.C. Kao
UCLA ECE TAs: S. Athreya, Y. Chang, S. Dong, K. Pang, M. Turali, L. Zou

Due Monday, 26 Jan 2025, by 11:59pm to Gradescope.
100 points total.

1. (10 points) **Noisy linear regression**

   A real estate company have assigned us the task of building a model to predict the house prices in Westwood. For this task, the company has provided us with a dataset $\mathcal{D}$:

   $$\mathcal{D} = \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \ldots, (x^{(N)}, y^{(N)})\}$$

   where $x^{(i)} \in \mathbb{R}^d$ is a feature vector of the ith house and $y^{(i)} \in \mathbb{R}$ is the price of the ith house. Since we just learned about linear regression, so we have decided to use a linear regression model for this task. Additionally, the IT manager of the real estate company have requested us to design a model with small weights. In order to accommodate his request, we will design a linear regression model with parameter regularization. In this problem, we will navigate through the process of achieving regularization by adding noise to the feature vectors. Recall, that we define the cost function in a linear regression problem as:

   $$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N}(y^{(i)} - (x^{(i)})^T\theta)^2$$

   where $\theta \in \mathbb{R}^d$ is the parameter vector. As mentioned earlier, we will be adding noise to the feature vectors in the dataset. Specifically, we will be adding zero-mean gaussian noise of known variance σ^2 from the distribution

   $$\mathcal{N}(0, \sigma^2 I)$$

   where $I \in \mathbb{R}^{d\times d}$ and $\sigma \in \mathbb{R}$. With the addition of gaussian noise the modified cost function is given by,

   $$\tilde{\mathcal{L}}(\theta) = \frac{1}{N}\sum_{i=1}^{N}(y^{(i)} - (x^{(i)} + \delta^{(i)})^T\theta)^2$$

   where $\delta^{(i)}$ are i.i.d noise vectors with $\delta^{(i)} \sim \mathcal{N}(0, \sigma^2 I)$.

   (a) (6 points) Express the expectation of the modified loss over the gaussian noise, $\mathbb{E}_{\delta\sim\mathcal{N}}[\tilde{\mathcal{L}}(\theta)]$, in terms of the original loss plus a term independent of the data $\mathcal{D}$. To be precise, your answer should be of the form:

   $$\mathbb{E}_{\delta\sim\mathcal{N}}[\tilde{\mathcal{L}}(\theta)] = \mathcal{L}(\theta) + R$$

where R is not a function of $\mathcal{D}$. For answering this part, you might find the following result useful:

$$\mathbb{E}_{\delta\sim\mathcal{N}}[\delta\delta^T] = \sigma^2 I$$

Solution:

$$\begin{aligned}
\mathbb{E}_{\delta\sim\mathcal{N}}[\tilde{\mathcal{L}}(\theta)] &= \frac{1}{N}\sum_{i=1}^{N}\mathbb{E}_{\delta\sim\mathcal{N}}[(y^{(i)} - (x^{(i)} + \delta^{(i)})^T\theta)^2] \\
&= \frac{1}{N}\sum_{i=1}^{N}\mathbb{E}_{\delta\sim\mathcal{N}}[(y^{(i)} - {x^{(i)}}^T\theta)^2] - 2\mathbb{E}_{\delta\sim\mathcal{N}}[(y^{(i)} - {x^{(i)}}^T\theta)({\delta^{(i)}}^T\theta)] + \mathbb{E}_{\delta\sim\mathcal{N}}[({\delta^{(i)}}^T\theta)^2].
\end{aligned}$$

Now,

$$\begin{aligned}
\mathbb{E}_{\delta\sim\mathcal{N}}[(y^{(i)} - {x^{(i)}}^T\theta)^2] &= (y^{(i)} - {x^{(i)}}^T\theta)^2 \\
\mathbb{E}_{\delta\sim\mathcal{N}}[(y^{(i)} - {x^{(i)}}^T\theta)({\delta^{(i)}}^T\theta)] &= 0 \\
\mathbb{E}_{\delta\sim\mathcal{N}}[({\delta^{(i)}}^T\theta)^2] &= \sigma^2\|\theta\|_2^2.
\end{aligned}$$

Therefore,

$$\mathbb{E}_{\delta\sim\mathcal{N}}[\tilde{\mathcal{L}}(\theta)] = \frac{1}{N}\sum_{i=1}^{N}(y^{(i)} - {x^{(i)}}^T\theta)^2 + \sigma^2\|\theta\|_2^2.$$

(b) (2 points) Based on your answer to (a), under expectation what regularization effect would the addition of the noise have on the model?

Solution: From (a), we can observe that noise would have a L_2 regularization effect on the model with regularization strength σ.

(c) (1 point) Suppose $\sigma \longrightarrow 0$, what effect would this have on the model?

Solution: As the regularization strength $\sigma \longrightarrow 0$, then we have no regularization and hence the model might overfit the data.

(d) (1 point) Suppose $\sigma \longrightarrow \infty$, what effect would this have on the model?

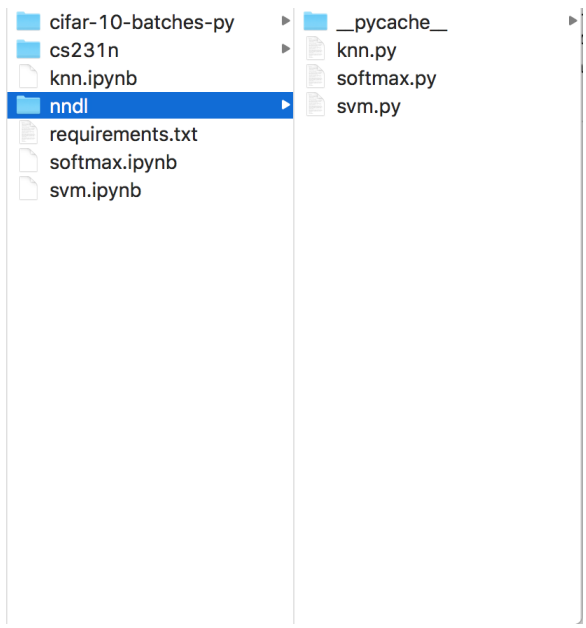
Solution: As the regularization strength $\sigma \longrightarrow \infty$, then the objective of the cost function is to minimize the L_2 norm of the parameter vector θ and hence $\theta \longrightarrow 0$ and the model will underfit the data.

2. (20 points) **k-nearest neighbors.** Complete the k-nearest neighbors Jupyter notebook. The goal of this workbook is to give you experience with the CIFAR-10 dataset, training and evaluating a simple classifier, and k-fold cross validation. In the Jupyter notebook, we'll be using the CIFAR-10 dataset. Acquire this dataset by running:

```
wget http://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
tar -xzvf cifar-10-python.tar.gz
rm cifar-10-python.tar.gz
```

If you don't have `wget` you can simply go to: `https://www.cs.toronto.edu/~kriz/cifar.html` and download it manually.

We have attached a screenshot of the paths the files ought to be in, in case helpful (though it should be apparent from the Jupyter notebook).



Print out the entire workbook and related code sections in knn.py, then submit them as a pdf to gradescope.

Solution: As part of the policy, we don't release solution to the coding component.

3. (30 points) **Softmax classifier gradient.** For softmax classifier, derive the gradient of the log likelihood.

   Concretely, assume a classification problem with c classes

   - Samples are $(\mathbf{x}^{(1)}, y^{(1)}), \ldots, (\mathbf{x}^{(m)}, y^{(m)})$, where $\mathbf{x}^{(j)} \in \mathbb{R}^n$, $y^{(j)} \in \{1, \ldots, c\}$, $j = 1, \ldots, m$
   - Parameters are $\theta = \{\mathbf{w}_i, b_i\}_{i=1,\ldots,c}$
   - Probablistic model is

     $$\Pr\left(y^{(j)} \mid \mathbf{x}^{(j)}, \theta\right) = \mathrm{softmax}_{y^{(j)}}(\mathbf{x}^{(j)})$$

     where

     $$\mathrm{softmax}_{y^{(j)}}(\mathbf{x}^{(j)}) = \frac{e^{\mathbf{w}_{y^{(j)}}^T \mathbf{x}^{(j)} + b_{y^{(j)}}}}{\sum_{k=1}^{c} e^{\mathbf{w}_k^T \mathbf{x}^{(j)} + b_k}}$$

Derive the log-likelihood $\mathcal{L}$, and its gradient w.r.t. the parameters, $\nabla_{\mathbf{w}_i}\mathcal{L}$ and $\nabla_{b_i}\mathcal{L}$, for $i = 1, \ldots, c$.

Note: We can group $\mathbf{w}_i$ and b_i into a single vector by augmenting the data vectors with an additional dimension of constant 1. Let $\tilde{\mathbf{x}} = \begin{bmatrix}\mathbf{x}\\1\end{bmatrix}$, $\tilde{\mathbf{w}}_i = \begin{bmatrix}\mathbf{w}_i\\b_i\end{bmatrix}$, then $a_i(\mathbf{x}) = \mathbf{w}_i^T\mathbf{x} + b_i = \tilde{\mathbf{w}}_i^T\tilde{\mathbf{x}}$. This unifies $\nabla_{\mathbf{w}_i}\mathcal{L}$ and $\nabla_{b_i}\mathcal{L}$ into $\nabla_{\tilde{\mathbf{w}}_i}\mathcal{L}$.

Solution:

- The data log-likelihood $\mathcal{L}$:

$$\begin{aligned}
\mathcal{L} &= \log \prod_{j=1}^{m} \Pr\left(y^{(j)} \mid \mathbf{x}^{(j)}, \theta\right) \\
&= \sum_{j=1}^{m} \log\left(\mathrm{softmax}_{y^{(j)}}\left(\mathbf{x}^{(j)}\right)\right) \\
&= \sum_{j=1}^{m} \log \frac{e^{\mathbf{w}_{y^{(j)}}^T \mathbf{x}^{(j)} + b_{y^{(j)}}}}{\sum_{k=1}^{c} e^{\mathbf{w}_k^T \mathbf{x}^{(j)} + b_k}} \\
&= \sum_{j=1}^{m} \left(\left(\mathbf{w}_{y^{(j)}}^T \mathbf{x}^{(j)} + b_{y^{(j)}}\right) - \log \sum_{k=1}^{c} e^{\mathbf{w}_k^T \mathbf{x}^{(j)} + b_k}\right) \qquad (1)
\end{aligned}$$

- The gradient w.r.t. $\mathbf{w}_i$, i.e., $\nabla_{\mathbf{w}_i}\mathcal{L}$: Because of the linearity of gradient operator, it is sufficient to study the jth terms of the summation in (1) individually.

$$\begin{aligned}
\nabla_{\mathbf{w}_i}\left(\left(\mathbf{w}_{y^{(j)}}^T \mathbf{x}^{(j)} + b_{y^{(j)}}\right) - \log \sum_{k=1}^{c} e^{\mathbf{w}_k^T \mathbf{x}^{(j)} + b_k}\right) &= \begin{cases} \mathbf{x}^{(j)} - \frac{e^{\mathbf{w}_i^T \mathbf{x}^{(j)} + b_i}}{\sum_{k=1}^{c} e^{\mathbf{w}_k^T \mathbf{x}^{(j)} + b_k}} \mathbf{x}^{(j)} & y^{(j)} = i \\ -\frac{e^{\mathbf{w}_i^T \mathbf{x}^{(j)} + b_i}}{\sum_{k=1}^{c} e^{\mathbf{w}_k^T \mathbf{x}^{(j)} + b_k}} \mathbf{x}^{(j)} & \text{otherwise} \end{cases} \\
&= \begin{cases} \left(1 - \mathrm{softmax}_i\left(\mathbf{x}^{(j)}\right)\right)\mathbf{x}^{(j)} & y^{(j)} = i \\ -\mathrm{softmax}_i\left(\mathbf{x}^{(j)}\right)\mathbf{x}^{(j)} & \text{otherwise} \end{cases}
\end{aligned}$$

  Therefore

$$\nabla_{\mathbf{w}_i}\mathcal{L} = \sum_{j=1}^{m} \left(\mathbb{1}_{\{y^{(j)}=i\}} - \mathrm{softmax}_i\left(\mathbf{x}^{(j)}\right)\right)\mathbf{x}^{(j)} \qquad (2)$$

  where $\mathbb{1}_{\{q\}}$ is the indicator function of a proposition q, which takes the value 1 if the proposition q is true, and 0 otherwise.

- Similar to (2), we have

$$\nabla_{\mathbf{b}_i}\mathcal{L} = \sum_{j=1}^{m} \left(\mathbb{1}_{\{y^{(j)}=i\}} - \mathrm{softmax}_i\left(\mathbf{x}^{(j)}\right)\right) \qquad (3)$$

4. (10 points) **Hinge loss gradient.**

Due to the drastic changes in climate throughout the world, a weather forecasting organization wants our help to build a model that can classify the observed weather patterns as severe or not severe. They have accumulated data on various attributes of the weather pattern such as temperature, precipitation, humidity, wind speed, air pressure, and geographical location along with severity of weather. However, the contribution of the attributes to the classification of weather as severe or not is unknown.

We choose to use a binary support vector machine (SVM) classification model. The SVM model parameters are learned by optimizing a hinge loss. The company has provided us with a data-set

$$\mathcal{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \cdots, (\mathbf{x}^{(K)}, y^{(K)})\}$$

where $\mathbf{x}^{(i)} \in \mathbb{R}^d$ is a feature vector of the ith data sample and $y^{(i)} \in \{-1, 1\}$. We define the hinge loss per training sample as

$$\text{hinge}_{y^{(i)}}(\mathbf{x}^{(i)}) = \max\left(0, 1 - y^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b)\right) \tag{4}$$

, where $\mathbf{w} \in \mathbb{R}^d$ and bias $b \in \mathbb{R}$ are the model parameters. With the hinge loss per sample defined, we can then formulate the average loss for our model as:

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{K}\sum_{i=1}^{K} \text{hinge}_{y^{(i)}}(\mathbf{x}^{(i)}) \tag{5}$$

Find the gradient of the loss function $\mathcal{L}(\mathbf{w}, b)$ with respect to the parameters i.e $\nabla_{\mathbf{w}}\mathcal{L}$ and $\nabla_b\mathcal{L}$.

Hint: An Indicator function, also known as a characteristic function, takes on the value of 1 at certain designated points and 0 at all other points. Mathematically, we can represent it as follows:

$$\mathbb{1}_{\{p<1\}} = \begin{cases} 1, & \text{if } p < 1 \\ 0, & \text{otherwise} \end{cases} \tag{6}$$

Solution: The loss function is given by:

$$\begin{aligned} \mathcal{L}(w, b) &= \frac{1}{K}\sum_{i=1}^{K} \text{hinge}_{y^{(i)}}(x^{(i)}) \\ &= \frac{1}{K}\sum_{i=1}^{K} max(0, 1 - y^{(i)}(w^T x^{(i)} + b)) \end{aligned}$$

The Hinge-loss function can be written as follows:

$$\begin{aligned} \text{hinge}_{y^{(i)}}(x^{(i)}) &= \max\left(0, 1 - y^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b)\right) \\ &= \begin{cases} 1 - y^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b), & \text{if } y^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b) < 1 \\ 0, & \text{otherwise} \end{cases} \end{aligned}$$

Finding gradient $\nabla_{\mathbf{w}}\mathcal{L}$ is

$$\begin{aligned}\nabla_{\mathbf{w}}\mathcal{L} &= \frac{1}{K}\sum_{i=1}^{K}\nabla_{\mathbf{w}}\text{hinge}_{y^{(i)}}(\mathbf{x}^{(i)}) \\ &= \frac{1}{K}\sum_{i=1}^{K}\left[\mathbb{1}_{\{y^{(i)}(\mathbf{w}^T x^{(i)}+b)<1\}}\left(-y^{(i)}\mathbf{x}^{(i)}\right)\right]\end{aligned}$$

Finding gradient $\nabla_b\mathcal{L}$ is

$$\begin{aligned}\nabla_b\mathcal{L} &= \frac{1}{K}\sum_{i=1}^{K}\nabla_b\text{hinge}_{y^{(i)}}(\mathbf{x}^{(i)}) \\ &= \frac{1}{K}\sum_{i=1}^{K}\left[\mathbb{1}_{\{y^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)}+b)<1\}}\left(-y^{(i)}\right)\right]\end{aligned}$$

where

$$\mathbb{1}_{\{y^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)}+b)<1\}} = \begin{cases}1, & \text{if } y^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)}+b)<1 \\ 0, & \text{otherwise}\end{cases}$$

5. (30 points) **Softmax classifier.** Complete the Softmax Jupyter notebook. Print out the entire workbook and related code sections in softmax.py, then submit them as a pdf to gradescope.